



RIFA.GG

GUIA COMPLETO DE INSTALACAO

GitHub + Vercel + Supabase

Passo a passo detalhado para colocar a versao limpa do projeto no ar, configurar o banco PostgreSQL, as variaveis de ambiente e o fluxo de deploy.

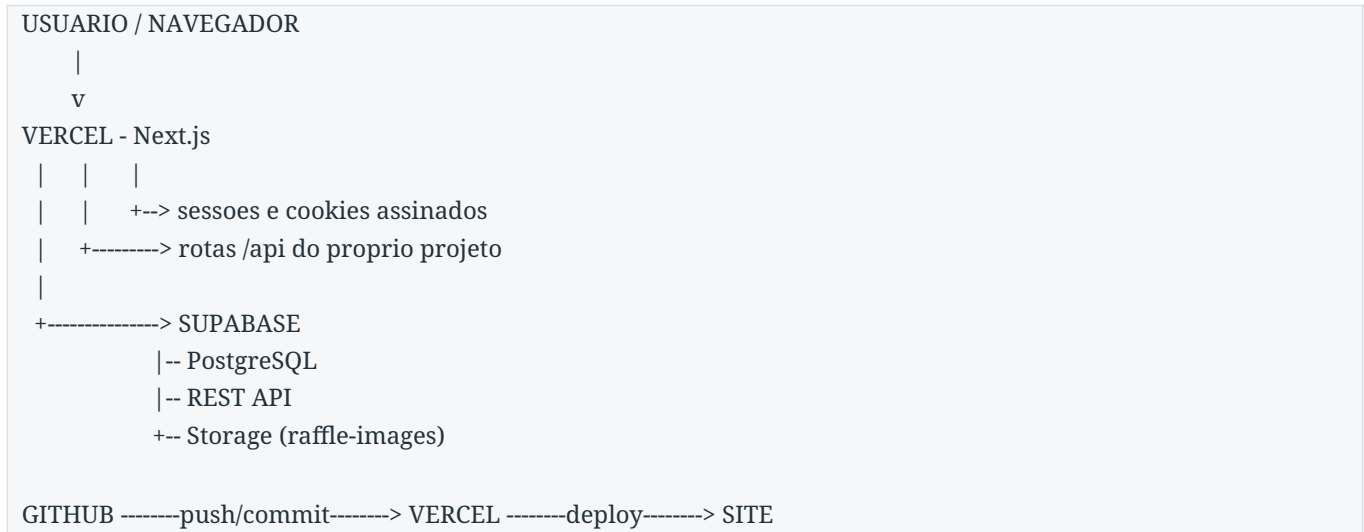
Projeto analisado: Next.js 16.3.4 | TypeScript 5.9.3 | React 19.2.6 | Node.js >= 22

Objetivo deste PDF

Levar uma copia nova do RIFA.GG do arquivo ZIP ate uma instalacao funcional, entendendo o papel do GitHub, da Vercel e do Supabase. O guia usa os nomes de arquivos e variaveis existentes na versao limpa do projeto.

1. Visao geral da arquitetura

Antes de configurar qualquer conta, entenda o caminho dos dados. Nesta versao do RIFA.GG, o navegador acessa a aplicacao Next.js hospedada na Vercel. O codigo do servidor faz chamadas ao Supabase usando uma chave secreta armazenada apenas nas variaveis de ambiente da Vercel.



Importante sobre autenticacao

O projeto nao usa o Supabase Auth como mecanismo principal de login. A autenticacao do RIFA.GG e implementada pelo proprio codigo: contas ficam na tabela public.accounts e as sessoes sao assinadas com SESSION_SECRET. O Supabase fornece principalmente PostgreSQL, REST API e Storage.

O que cada servico faz

Servico	Funcao no RIFA.GG
GitHub	Armazena o codigo-fonte, historico de commits e serve como origem para os deploys.
Vercel	Hospeda o Next.js, executa as rotas do servidor, recebe as variaveis secretas e publica Production/Preview.
Supabase	Hospeda o PostgreSQL, expoe a REST API usada pelo servidor e armazena imagens no bucket raffle-images.

2. O que voce precisa antes de comecar

- Uma conta no GitHub.
- Uma conta na Vercel, de preferencia conectada ao mesmo GitHub.
- Uma conta no Supabase.
- O ZIP limpo do RIFA.GG extraido em uma pasta do computador.
- Git instalado para enviar o projeto pelo terminal.
- Node.js 22 ou superior. O package.json do projeto declara Node >= 22.
- Um editor como Visual Studio Code e um navegador atualizado.

Estrutura importante do projeto

```
RIFA.GG/  
|-- app/  
|-- components/  
|-- lib/  
|-- public/  
|-- supabase/  
|   |-- schema.sql  
|   |-- latest-update.sql  
|   +-- uma-conta-por-ip.sql  
|-- .env.example  
|-- .gitignore  
|-- package.json  
|-- package-lock.json  
+-- next.config.ts
```

Qual SQL usar?

Instalacao NOVA e vazia: execute somente supabase/schema.sql. O proprio arquivo diz que e o banco novo e vazio. Banco EXISTENTE com dados: use supabase/latest-update.sql. O arquivo uma-conta-por-ip.sql e um patch especifico; na versao nova do schema a regra de IP ja esta incorporada, portanto nao e necessario executa-lo separadamente em uma instalacao nova.

3. Criar o projeto no Supabase

1. Entre no painel do Supabase e escolha a organizacao onde deseja criar o projeto.
2. Clique para criar um novo projeto.
3. Defina um nome, por exemplo RIFA-GG-TESTE ou RIFA-GG-PROD.
4. Escolha uma senha forte para o banco e guarde-a em um gerenciador de senhas.
5. Escolha a regioao mais adequada para os usuarios do projeto.
6. Conclua a criacao e aguarde o banco ficar disponivel antes de executar o SQL.

Recomendacao

Para estudo, crie primeiro um projeto Supabase separado de qualquer ambiente que voce considere importante. Para Preview/Staging, prefira outro projeto de banco em vez de apontar testes para o mesmo banco de Production.

3.1 Executar o banco novo

7. No Supabase, abra SQL Editor.
8. Crie uma nova query.
9. No computador, abra supabase/schema.sql.
10. Copie o arquivo inteiro, cole no SQL Editor e clique em Run.
11. Espere a execucao terminar. Se houver erro, nao avance ate entender a mensagem.

O schema do projeto cria tabelas, funcoes, restricoes, permissoes e o bucket de imagens utilizado pela rifa.

3.2 Conferir se o schema entrou

No Table Editor, confirme pelo menos estas tabelas:

- accounts
- raffles
- raffle_numbers
- purchases
- coupons
- audit_logs

Voce tambem pode executar estas consultas no SQL Editor:

```
select tablename
from pg_tables
where schemaname = 'public'
order by tablename;
```

```
select id, name, public
from storage.buckets
where id = 'raffle-images';
```

Se o bucket raffle-images nao aparecer, revise a primeira mensagem de erro no SQL Editor. Isso normalmente significa que o schema nao terminou por completo; evite criar o bucket manualmente antes de entender o erro.

4. Pegar a URL e a chave do Supabase

O código desta versão espera `SUPABASE_URL` e `SUPABASE_SECRET_KEY`. O Supabase atualmente recomenda chaves secretas no formato `sb_secret_...` para componentes de servidor. Essa chave tem privilégios elevados e nunca deve ser enviada ao navegador ou colocada em arquivo público.

12. No projeto Supabase, abra o painel Connect ou a área Settings > API Keys.
13. Copie a Project URL. Ela se parece com `https://SEU-PROJETO.supabase.co`.
14. Localize ou crie uma Secret key no formato `sb_secret_...`
15. Guarde a chave em local seguro. Não cole em README, GitHub, print ou código-fonte.

NAO use NEXT_PUBLIC_

Neste projeto a chave é usada pelo servidor Next.js. Não renomeie `SUPABASE_SECRET_KEY` para `NEXT_PUBLIC_SUPABASE_SECRET_KEY`. Variáveis `NEXT_PUBLIC` podem ser incorporadas ao código entregue ao navegador.

O código ainda aceita `SUPABASE_SERVICE_ROLE_KEY` como compatibilidade antiga, mas a configuração recomendada para uma instalação nova é `SUPABASE_SECRET_KEY`.

5. Configurar o arquivo .env.local no computador

Na raiz do projeto existe .env.example. Copie esse arquivo para .env.local. O .gitignore ja deve impedir que .env.local seja enviado ao GitHub.

```
# PowerShell - dentro da pasta do projeto
Copy-Item .env.example .env.local
```

Variaveis usadas pelo RIFA.GG

Variavel	Obrigatoria?	O que colocar
SUPABASE_URL	Sim	A Project URL do Supabase.
SUPABASE_SECRET_KEY	Sim	A chave secreta sb_secret_... usada somente no servidor.
ADMIN_USERNAME	Recomendado	Usuario usado para criar o primeiro owner quando ainda nao existe admin no banco.
ADMIN_PASSWORD	Sim	Senha forte do bootstrap administrativo.
SESSION_SECRET	Sim	Segredo com pelo menos 32 caracteres para assinar as sessoes.
DISCORD_WEBHOOK_URL	Nao	Webhook opcional para avisos de sorteio e teste da integracao Discord.

Exemplo sem valores reais

```
SUPABASE_URL=https://SEU-PROJETO.supabase.co
SUPABASE_SECRET_KEY=sb_secret_COLOQUE_AQUI
ADMIN_USERNAME=seu_admin
ADMIN_PASSWORD=UMA_SENHA_FORTE
SESSION_SECRET=UM_SEGREDO_LONGO_COM_PELo_MENOS_32_CARACTERES
DISCORD_WEBHOOK_URL=
```

Gerar um SESSION_SECRET forte

Com Node.js instalado, rode:

```
node -e "console.log(require('crypto').randomBytes(32).toString('hex'))"
```

Copie a saida e use como SESSION_SECRET. Nao reutilize senha pessoal.

6. Testar localmente antes do GitHub

Como existe package-lock.json, use npm ci para instalar exatamente as dependencias travadas no projeto.

```
npm ci
npm run lint
npm run build
npm run dev
```

Neste projeto, o script chamado lint executa tsc --noEmit, ou seja, faz uma verificacao de tipos TypeScript. Depois de npm run dev, abra:

- <http://localhost:3000> - pagina publica
- <http://localhost:3000/conta> - area de membro
- <http://localhost:3000/admin> - painel administrativo

6.1 Primeiro login administrativo

Se o banco estiver totalmente vazio e ainda nao existir nenhuma conta owner/admin/sponsor, o primeiro login em /admin usa ADMIN_USERNAME e ADMIN_PASSWORD. Se as credenciais estiverem corretas, o sistema cria a primeira conta com cargo owner no banco.

Depois que o primeiro admin existe

ADMIN_USERNAME e ADMIN_PASSWORD nao funcionam como uma senha universal. A partir dai o login administrativo consulta as contas armazenadas no banco. Portanto, trocar apenas as variaveis de bootstrap nao substitui a senha de um owner que ja existe.

6.2 Teste minimo local

16. Abra a pagina inicial e confirme que nao aparece erro 500.
17. Entre em /admin e crie o primeiro owner.
18. Crie uma rifa de teste no painel.
19. Envie uma imagem e confirme que ela aparece; isso testa o Storage raffle-images.
20. Crie uma conta de membro em /conta.
21. Reserve numeros em uma rifa de teste.
22. Volte ao painel e confirme que a compra aparece para revisao.

7. Preparar o projeto para o GitHub

Antes do primeiro commit, confirme que nenhuma credencial real sera enviada.

```
git status  
git check-ignore .env.local
```

O segundo comando deve indicar que .env.local esta ignorado. Abra tambem .gitignore e confirme que arquivos .env reais continuam bloqueados.

Se uma chave ja foi publicada

Apagar o arquivo em um commit novo nao torna a chave segura. Revogue/rotacione a chave no servico correspondente e depois limpe o historico, se necessario.

8. Criar o repositório no GitHub

8.1 Criar o repositório vazio

23. Entre no GitHub.
24. No menu de criação, escolha New repository.
25. Nome sugerido: RIFA.GG ou rifa-gg.
26. Escolha Public se a intenção for portfólio; Private se ainda estiver revisando.
27. Como o projeto já possui README e .gitignore, crie o repositório vazio para evitar conflito inicial.

8.2 Enviar pelo terminal

Abra o PowerShell na pasta onde está package.json e execute:

```
git init
git add .
git commit -m "Publica versão de estudo do RIFA.GG"
git branch -M main
git remote add origin https://github.com/SEU_USUARIO/RIFA.GG.git
git push -u origin main
```

Troque SEU_USUARIO pelo nome da sua conta e use a URL exata mostrada pelo GitHub. Dependendo da configuração do Git, a autenticação pode abrir o navegador ou usar o GitHub Credential Manager.

8.3 Conferência depois do push

- README aparece na página inicial do repositório.
- Pasta supabase contém os SQLs de teste.
- .env.local NÃO aparece.
- Nenhuma chave sb_secret_, senha, webhook ou token real aparece na busca do repositório.

9. Conectar o GitHub a Vercel

28. Entre na Vercel e use Continue with GitHub ou conecte sua conta GitHub existente.
29. Crie um novo Project e escolha Import Git Repository.
30. Selecione o repositório RIFA.GG.
31. Autorize a integração da Vercel ao repositório se for solicitado.
32. A Vercel deve detectar Next.js automaticamente.
33. Mantenha Root Directory na raiz do repositório, pois package.json está na raiz.
34. Não clique em Deploy antes de configurar as variáveis de ambiente, ou o site poderá subir com erro de conexão.

Configuração de build

O package.json possui build: next build e start: next start. Em um projeto Next.js importado normalmente, a Vercel detecta esses comandos sem necessidade de alterar Build Command ou Output Directory.

10. Configurar as variaveis na Vercel

Dentro do projeto Vercel, abra Settings > Environment Variables. Cadastre os mesmos nomes do .env.local.

Nome	Ambiente recomendado	Observacao
SUPABASE_URL	Production	URL do projeto Supabase de producao.
SUPABASE_SECRET_KEY	Production / Sensitive	Chave secreta do servidor. Nunca expor em cliente.
ADMIN_USERNAME	Production	Usuario de bootstrap do primeiro owner.
ADMIN_PASSWORD	Production / Sensitive	Senha forte do bootstrap.
SESSION_SECRET	Production / Sensitive	Segredo longo para cookies e sessoes.
DISCORD_WEBHOOK_URL	Production / Sensitive (opcional)	Deixe ausente se nao quiser integrar o Discord.

Preview exige cuidado

Branches diferentes da branch de producao podem gerar Preview Deployments. Se voce colocar as mesmas chaves de Production em Preview, os testes de uma branch poderao alterar o mesmo banco real. O ideal e usar outro projeto Supabase para Preview ou nao habilitar essas variaveis em Preview enquanto estiver estudando.

Depois de adicionar ou alterar variaveis, faca um novo deploy. A Vercel informa que alteracoes de Environment Variables nao retroagem para deploys antigos.

11. Fazer o primeiro deploy

- 35. Com as variaveis cadastradas, clique em Deploy ou Redeploy.
- 36. Acompanhe o Build Logs.
- 37. Espere o status Ready.
- 38. Abra a URL gerada pela Vercel.
- 39. Teste /admin, /conta e a pagina publica.

O fluxo completo depois disso

```
VOCE ALTERA O CODIGO
|
v
git add / git commit / git push
|
v
GITHUB recebe o commit
|
v
VERCEL inicia um novo deploy
|
v
Next.js e publicado
|
v
Aplicacao usa SUPABASE_URL + SUPABASE_SECRET_KEY no servidor
|
v
PostgreSQL / REST / Storage no Supabase
```

Com a integracao GitHub ativa, pushes em branches geram deploys conforme a configuracao da Vercel; normalmente a branch de producao (geralmente main) atualiza Production e outras branches geram Preview.

12. Checklist de validacao em producao

Teste	Resultado esperado
Home	Carrega a rifa/estado sem erro de banco.
Primeiro /admin	Cria o primeiro owner somente se ainda nao existir admin.
Login posterior	Usa a conta administrativa salva no banco.
Cadastro de membro	Cria conta valida e respeita as regras atuais do schema.
Imagem da rifa	Upload funciona no bucket raffle-images.
Reserva	Numeros ficam reservados e compra aparece no painel.
Aprovacao	Estado da compra e dos numeros muda corretamente.
Cupom	Validacao respeita percentual, validade, usos e minimo.
Discord	Se configurado, o botao de teste envia uma mensagem.

Teste com dados descartaveis

Antes de convidar pessoas reais, faca testes de abuso: IDs invalidos, varias contas, repeticao de requisicoes, concorrencia de reservas, permissoes de cargos e tentativas de acessar rotas administrativas sem sessao.

13. Como atualizar o projeto depois

Atualizacao de codigo

```
git add .  
git commit -m "Descreva a alteracao"  
git push
```

O push atualiza o GitHub e a Vercel cria um novo deploy automaticamente quando a integracao esta ativa.

Atualizacao de banco

Nao misture arquivos SQL sem saber o estado do banco:

- Banco novo e vazio: schema.sql.
- Banco existente com dados: latest-update.sql, conforme o comentario do proprio arquivo.
- Patch uma-conta-por-ip.sql: use apenas se voce estiver aplicando essa correcao a uma instalacao antiga que ainda nao a possui.

Backup primeiro

Antes de aplicar alteracoes SQL em um banco que ja possui dados, faca backup/exportacao e teste a atualizacao em outro ambiente quando possivel.

14. Problemas comuns e como diagnosticar

Sintoma	Causa provavel / acao
Erro: Configure SUPABASE_URL e SUPABASE_SECRET_KEY	Variavel ausente, nome errado ou deploy antigo. Corrija na Vercel e redeploy.
SESSION_SECRET precisa ter pelo menos 32 caracteres	Gere um novo segredo forte com 32+ caracteres e redeploy.
Primeiro admin nao entra	Confira ADMIN_USERNAME/ADMIN_PASSWORD. Se ja existe admin no banco, o bootstrap nao e mais usado.
Tabela/funcoes nao encontradas	schema.sql nao foi executado totalmente ou voce esta apontando para outro projeto Supabase.
Imagem nao envia	Confira se o bucket raffle-images existe e se SUPABASE_SECRET_KEY pertence ao mesmo projeto da URL.
Discord mostra NAO CONFIGURADO	DISCORD_WEBHOOK_URL nao foi configurada; isso e opcional e nao impede o site de funcionar.
Mudou variavel e nada mudou	Variaveis novas so entram em novos deploys; use Redeploy.
Preview alterou dados reais	Preview esta usando as credenciais do banco de Production. Separe os ambientes.
Build falha por Node	Use Node 22+; o projeto declara engines.node >=22.

15. Checklist de segurança antes de tornar público

- Nunca commitar `.env.local`.
- Nunca colocar `SUPABASE_SECRET_KEY` em variável `NEXT_PUBLIC`.
- Marcar segredos como Sensitive na Vercel quando a opção estiver disponível.
- Usar senha administrativa longa e única.
- Usar `SESSION_SECRET` aleatório e diferente da senha do admin.
- Rotacionar imediatamente qualquer chave que tenha sido publicada.
- Evitar que Preview e Production compartilhem o mesmo banco durante testes.
- Revisar SQL, permissões e funções security definir antes de considerar uso real.
- Manter backups e logs para entender problemas de produção.
- Tratar este repositório como projeto experimental até uma revisão técnica adequada.

Por que a chave do Supabase merece atenção extra

A Secret key do Supabase tem privilégios elevados e pode ignorar Row Level Security. Neste projeto ela deve permanecer exclusivamente no ambiente do servidor Next.js/Vercel. Quem obtiver essa chave pode ter acesso muito amplo ao banco.

16. Resumo rapido - ordem correta

40. Extrair o ZIP limpo.
41. Criar projeto novo no Supabase.
42. Executar supabase/schema.sql.
43. Copiar Project URL e Secret key.
44. Criar .env.local com todas as variaveis.
45. Rodar npm ci, npm run lint, npm run build e npm run dev.
46. Testar home, membro, admin, imagem e reserva localmente.
47. Criar repositorio vazio no GitHub e fazer push.
48. Importar o repositorio na Vercel.
49. Adicionar as variaveis de ambiente na Vercel.
50. Fazer Deploy/Redeploy.
51. Entrar em /admin para criar o primeiro owner no banco vazio.
52. Executar checklist de producao antes de convidar usuarios.

17. Referencias oficiais consultadas

Este guia foi preparado em 24/09/2026 e combina a estrutura real do ZIP do RIFA.GG com a documentacao oficial atual dos servicos. Interfaces podem mudar com o tempo; se um nome de menu estiver diferente, consulte a documentacao oficial abaixo.

[GitHub - Creating a new repository](#)

[GitHub - Adding locally hosted code to GitHub](#)

[Vercel - Deploying GitHub Projects with Vercel](#)

[Vercel - Environment Variables](#)

[Supabase - API keys](#)

[Supabase - Database overview / SQL Editor](#)

[Next.js - Deploying](#)

Arquivos do RIFA.GG usados como base

- package.json - versoes, scripts e requisito Node >=22.
- .env.example - nomes das variaveis de ambiente.
- lib/supabase.ts - leitura de SUPABASE_URL/SUPABASE_SECRET_KEY e chamadas REST/Storage.
- lib/auth.ts e app/api/auth/login/route.ts - sessoes e bootstrap do primeiro owner.
- supabase/schema.sql - instalacao nova e vazia.
- supabase/latest-update.sql - atualizacao de instalacoes existentes.
- supabase/uma-conta-por-ip.sql - patch especifico para instalacoes antigas.

Fim do guia

Se todos os passos estiverem verdes, o fluxo GitHub -> Vercel -> Supabase esta conectado. A proxima etapa deve ser testar seguranca e comportamento real em um ambiente de teste antes de considerar qualquer uso serio.